How-To Library

Informatica

Enabling and Configuring PowerExchange Data Maps Caching

© Copyright Informatica LLC 2010, 2021. Informatica

Abstract

When you enable PowerExchange data maps caching, PowerExchange retrieves data maps from job-level memory rather than by accessing the data maps file, which improves performance. This article shows how to enable and configure PowerExchange data maps caching.

Supported Versions

- PowerExchange 10.0 or later

Table of Contents

Overview of Data Maps Caching. 2
Enabling Data Maps Caching. 3
Configuring Data Maps Caching to Run in Single-Job or Multiple-Jobs Mode. 3
Single-Job Mode. 3
Multiple-Jobs Mode. 3
Configuring Data Maps Caching - Example. 4
Reference Information for DBMOVER Statements for Data Maps Caching. 5
DMXCACHE_DELETEECSA Statement. 5
DMXCACHE_MAX_MEMORY_MB Statement. 6
DMXCACHE_MULTIPLEJOBS Statement. 6

Overview of Data Maps Caching

You can configure PowerExchange to cache data maps defined for nonrelational z/OS data sources on the system where the data maps are used by a single PowerExchange Listener job or by multiple PowerExchange Listener, netport, and batch jobs.

When you enable data maps caching, PowerExchange retrieves data maps from job-level memory rather than by accessing the data maps file. By eliminating accesses to the data maps file, data maps caching improves performance.

When you enable data maps caching, you can decrease initialization time by 0.1 second in addition to the performance improvements that you achieve through connection pooling. For more information about connection pooling, see the *PowerExchange Bulk Data Movement Guide*.

Data maps caching can be useful when many short transactions are completed for each second. However, if you run long-running bulk data movement sessions and initialization time is a small part of the overall processing time, data maps caching might not give you any performance benefits.

To enable and configure data maps caching, define DBMOVER statements on the system where the PowerExchange Listener or netport jobs run. You can configure PowerExchange to run data maps caching in either single-job mode or multiple-jobs mode.

Enabling Data Maps Caching

To enable data maps caching, set the DMXCACHE_MAX_MEMORY_MB statement to a value greater than 0 in the DBMOVER configuration file.

This value configures the maximum size for the data maps cache in megabytes.

For information about the DMXCACHE_MAX_MEMORY_MB statement, see "DMXCACHE_MAX_MEMORY_MB Statement" on page 6.

Note: Verify that the REGION size specified in the DTLLST step in the JCL is large enough to run the PowerExchange Listener job. For example, if you define DMXCACHE_MAX_MEMORY_MB=20, you might have to increase the REGION size by 20 MBs.

Configuring Data Maps Caching to Run in Single-Job or Multiple-Jobs Mode

You can configure PowerExchange to run data maps caching in either single-job mode or multiple-jobs mode.

If you use one PowerExchange Listener job and no netport jobs, configure PowerExchange to run data maps caching in single-job mode.

If you use multiple PowerExchange Listener or netport jobs to access the same data maps file, configure PowerExchange to run data maps caching in multiple-jobs mode.

Single-Job Mode

In single-job mode, PowerExchange maintains copies of previously read data maps in job-level memory. PowerExchange does not use ESCA memory to maintain information about data maps.

On subsequent reads of data maps, PowerExchange retrieves the data maps from job-level memory, which eliminates the overhead of enqueues, opens, points, reads, and closes of the data maps KSDS file.

When a data map changes or is deleted, PowerExchange deletes the copy of that data map in the cache. For changed data maps, PowerExchange does not add the data map to the cache again until the data map is actually used.

Single-job mode is more efficient than multiple-jobs mode because in single-job mode, the PowerExchange Listener job is the only job that updates the data maps file and the data maps cache does not become stale. Consequently, PowerExchange never needs to completely clear the cache like it does in multiple-jobs mode.

To configure PowerExchange to run data maps caching in single-job mode, define the following DBMOVER configuration statement:

```
DMXCACHE_MULTIPLEJOBS=N
```

For information about the DMXCACHE_MULTIPLEJOBS statement, see "DMXCACHE_MULTIPLEJOBS Statement" on page 6.

Multiple-Jobs Mode

In multiple-jobs mode, PowerExchange maintains copies of previously read data maps in job-level memory. Additionally, when you start a PowerExchange Listener, PowerExchange dynamically allocates 4096 bytes of ECSA memory in which PowerExchange maintains the name of the data maps KSDS file and the time stamp of its last update. PowerExchange uses this information to determine whether the data maps cache is stale and needs to be cleared.

On subsequent reads of data maps, PowerExchange retrieves the data maps from job-level memory, which eliminates the overhead of enqueues, opens, points, reads, and closes of the data maps KSDS file.

When a data map changes or is deleted, PowerExchange completes the following processing:

- In ECSA memory, PowerExchange changes the time stamp of the last update to the data maps KSDS file.
- When another task tries to access any data map, PowerExchange determines if the cache is stale by comparing the time stamp in ECSA memory against the time stamp of the data maps cache. If the cache is stale, PowerExchange clears it and PowerExchange reads the required data map from disk and adds it to the cache.

To configure PowerExchange to run data maps caching in multiple-jobs mode, define the following DBMOVER configuration statement:

```
DMXCACHE_MULTIPLEJOBS=Y
```

Additionally, if you want PowerExchange to free the ECSA memory after the PowerExchange Listener closes, define the following DBMOVER configuration statement:

```
DMXCACHE_DELETEECSA=Y
```

However, because shared memory allocation is a complex task and 4096 bytes of memory is a small amount, you can accept the default value, which is N, to permanently retain the ECSA memory.

For information about the DMXCACHE_MULTIPLEJOBS statement, see "DMXCACHE_MULTIPLEJOBS Statement" on page 6. For information about the DMXCACHE_DELETEECSA statement, see "DMXCACHE_DELETEECSA Statement" on page 5.

Configuring Data Maps Caching - Example

In this example, you enable data maps caching and configure a maximum size of 20 MBs for the data maps cache, which is large enough to hold 20 large IDMS data maps or thousands of small data maps.

Verify that the REGION size specified in the DTLLST step in the JCL is large enough to run the PowerExchange Listener job. For example, if you define DMXCACHE_MAX_MEMORY_MB=20, you might have to increase the REGION size by 20 MBs.

1. To enable data maps caching and to configure a data maps cache size of 20 MBs, define the following statement in the DBMOVER configuration file:

   ```
   DMXCACHE_MAX_MEMORY_MB=20
   ```

2. Based on your environment, configure PowerExchange to run data maps caching in single-job or multiple-jobs mode.

Environment	Mode	Configuration Steps
You use one PowerExchange Listener job and no netport jobs.	Single-job mode	Define the following DBMOVER configuration statement: `DMXCACHE_MULTIPLEJOBS=N` In this situation, the default cache size is sufficient. Default is N. For more information about single-job mode, see "Single-Job Mode" on page 3.

You use multiple PowerExchange Listener or netport jobs to access the same data maps file.	Multiple-jobs mode	Define the following DBMOVER configuration statement:
		`DMXCACHE_MULTIPLEJOBS=Y`
		Additionally, if you want PowerExchange to free the ECSA memory when the PowerExchange Listener closes, define the following DBMOVER configuration statement:
		`DMXCACHE_DELETEECSA=Y`
		However, because shared memory allocation is a complex task and 4096 bytes of memory is a small amount, you can accept the default value, which is N, to permanently retain the ECSA memory.
		For more information about multiple-jobs mode, see "Multiple-Jobs Mode" on page 3.

3. Optionally, if you run data maps caching in multiple-jobs mode, you can run the PWXUDMX utility to view the ECSA memory.

   For more information about the PWXUDMX utility, see the *PowerExchange Utilities Guide*.

4. To determine if data maps caching improved performance, close the PowerExchange Listener and review messages in the PowerExchange log.

Reference Information for DBMOVER Statements for Data Maps Caching

Use the following information to define DBMOVER configuration statements to enable and configure data maps caching.

DMXCACHE_DELETEECSA Statement

The DMXCACHE_DELETEECSA statement determines whether PowerExchange frees ECSA memory when data maps caching runs in multiple-jobs mode and no files exist with nonzero use counts in ECSA memory.

Operating Systems: z/OS

Related Statements: DM_SUBTASK, DMXCACHE_MAX_MEMORY_MB, and DMXCACHE_MULTIPLEJOBS

Required: No

Syntax:

```
DMXCACHE_DELETEECSA={N|Y}
```

Valid Values:

- **N**. PowerExchange does not free ECSA memory. Because allocating shared memory is a complex task and 4096 bytes of memory is a small amount, you can accept the default value, which is N, to permanently retain the ECSA memory.
- **Y**. PowerExchange frees ECSA memory when data maps caching runs in multiple-jobs mode and no files exist with nonzero use counts in ECSA memory.

  To run data maps caching in multiple-jobs mode, set the DMXCACHE_MULTIPLEJOBS statement to Y.

Default is N.

DMXCACHE_MAX_MEMORY_MB Statement

The DMXCACHE_MAX_MEMORY_MB statement enables PowerExchange data maps caching and configures the maximum size for the data maps cache in megabytes.

Operating Systems: z/OS

Related Statements: DM_SUBTASK, DMXCACHE_DELETEECSA, and DMXCACHE_MULTIPLEJOBS

Required: No

Syntax:

```
DMXCACHE_MAX_MEMORY_MB={cache_size|0}
```

Valid Values: For the *cache_size* variable, enter the maximum size for the data maps cache in megabytes. Valid values are from 0 through 4095. You must enter a value greater than 0 to enable data maps caching. If you accept the default value of 0, PowerExchange does not use data maps caching.

Default is 0.

Note: Verify that the REGION size specified in the DTLLST step in the JCL is large enough to run the PowerExchange Listener job. For example, if you define DMXCACHE_MAX_MEMORY_MB=20, you might have to increase the REGION size by 20 MBs.

DMXCACHE_MULTIPLEJOBS Statement

The DMXCACHE_MULTIPLEJOBS statement determines whether PowerExchange runs data maps caching in multiple-jobs or single-job mode.

Operating Systems: z/OS

Related Statements: DM_SUBTASK, DMXCACHE_DELETEECSA, and DMXCACHE_MAX_MEMORY_MB

Required: No

Syntax:

```
DMXCACHE_MULTIPLEJOBS={N|Y}
```

Valid Values:

- **N**. PowerExchange runs data maps caching in single-job mode.
- **Y**. PowerExchange runs data maps caching in multiple-jobs mode.

Default is N.

Single-Job Mode: PowerExchange maintains copies of previously read data maps in job-level memory. On subsequent reads of data maps, PowerExchange retrieves the data maps from job-level memory, which eliminates the overhead of enqueues, opens, points, reads, and closes of the data maps KSDS file.

PowerExchange does not use ESCA memory to maintain information about data maps.

When a data map changes or is deleted, PowerExchange deletes the copy of that data map in the cache. For changed data maps, PowerExchange does not add the data map to the cache again until the data map is actually used.

Single-job mode is more efficient than multiple-jobs mode because in single-job mode, the PowerExchange Listener job is the only job that updates the data maps file and the data maps cache does not become stale. Consequently, PowerExchange never needs to completely clear the cache like it does in multiple-jobs mode.

Multiple-Jobs Mode: In multiple-jobs mode, PowerExchange maintains copies of previously read data maps in job-level memory. Additionally, when you start a PowerExchange Listener, PowerExchange dynamically allocates 4096 bytes of ECSA memory in which PowerExchange maintains the name of the data maps KSDS file and the time stamp of its last update. PowerExchange uses this information to determine whether the data maps cache is stale and needs to be cleared.

On subsequent reads of data maps, PowerExchange retrieves the data maps from job-level memory, which eliminates the overhead of enqueues, opens, points, reads, and closes of the data maps KSDS file.

When a data map changes or is deleted, PowerExchange completes the following processing:

- In ECSA memory, PowerExchange changes the time stamp of the last update to the data maps KSDS file.
- When another task tries to access any data map, PowerExchange determines if the cache is stale by comparing the time stamp in ECSA memory against the time stamp of the data maps cache. If the cache is stale, PowerExchange clears it, and then PowerExchange reads the required data map from disk and adds it to the cache.

When you close a PowerExchange Listener, PowerExchange frees the ECSA memory if the following conditions are true:

- The DMXCACHE_DELETEECSA statement is set to Y in the DBMOVER configuration file.
- No files exist with nonzero use counts in the ECSA memory.

When you run data maps caching in multiple-jobs mode, you can optionally use the PWXUDMX utility to manage the ECSA memory. For more information about the utility, see the *PowerExchange Utilities Guide*.

Author

Diane Fleming

Acknowledgements

The author would like to acknowledge Ross Ferrand.